

XAI - 4



Help me with my XAI (Explainable Artificial Intelligence) Assignment.
Answer the questions of the assignment which I will provide one at a time. Add proper updated information from your own knowledge or the internet to make the answer more complete and current.

Understood.

Proceed by sending the **first XAI assignment question**. I will give a **complete, updated, and exam-ready answer** with verified information (including current research or frameworks if relevant).



Unit 4 Chapter 4.1: XAI Methods

1. Classify XAI techniques by timing, scope, model dependency, and representation with examples.

Here is a structured, exam-ready classification of XAI (Explainable Artificial Intelligence) techniques according to the four requested dimensions: **timing**, **scope**, **model dependency**, and **representation**. I include definitions, distinctions, and examples.

1. Timing (when the explanation is generated)

Definition: This dimension classifies whether the explainability is built-into the model from the start (ante-hoc/intrinsic) or applied afterwards (post-hoc).

- **Intrinsic / Ante-hoc:** The model is designed to be interpretable by default. Explanations are part of the model architecture.
 - *Example:* A simple decision tree, linear regression, rule-based system. These models expose their logic (e.g. "if $X > 10$ then class = A") so the explanation is inherent. [MDPI+2](#)
- **Post-hoc:** A (possibly complex) model is trained first (often a "black-box" like a deep neural network or ensemble) and then an explanation technique is applied afterwards to interpret its decisions.
 - *Example:* Using LIME (Local Interpretable Model-agnostic Explanations) to explain a prediction of a deep network. Or computing SHAP values for a tree-ensemble model after training. [MDPI+1](#)

Key implication: Intrinsic methods sacrifice less "explanation effort" but often restrict model complexity (and perhaps performance). Post-hoc methods allow black-boxes but raise reliability/trust concerns (does the explanation truly reflect the model?).

2. Scope (what part of the model/decision is being explained)

Definition: This dimension identifies whether the explanation covers the whole model (global) or a single decision/instance (local).

- **Global explanation:** Explains the overall behaviour of the model (across many inputs), e.g., what features matter most overall, what patterns the model uses generally.
 - *Example:* Aggregated SHAP values showing that "age" and "income" are the top global predictors in a credit risk model. [MDPI+1](#)
- **Local explanation:** Explains a single prediction or small set of predictions (why did the model decide this output for this particular input?).
 - *Example:* LIME produces a local surrogate model around the input x_0 and explains that for this applicant the "income < 50k" and "credit scores < 600" led to rejection. [Springer Link+1](#)

Key implication: If you need to audit model-level behaviour (bias, feature importance, fairness) you use global methods; if you need to justify individual decisions (in e.g. medical diagnosis, credit lending) you use local methods.

3. Model dependency (whether the technique depends on internal model details or treats model as a black-box)

Definition: Classifies explainability methods into whether they are *model-specific* (depend on model internals) or *model-agnostic* (treat the model as a black box).

- **Model-specific approaches:** These exploit the internal structure of the model (weights, layers, architecture) for explanation. They may not apply to arbitrary model types.
 - *Examples:*
 - Attention maps in transformer networks (for NLP)
 - Grad-CAM (Gradient-weighted Class Activation Mapping) in CNNs for image classification. [AryaXAI+1](#)
 - TreeSHAP: specific efficient SHAP implementation for tree ensembles. [MDPI](#)
- **Model-agnostic approaches:** These treat the model as a black-box (you can only query it for input→output) and build explanations without knowing internals. Thus they can be applied across many model types.
 - *Examples:* LIME (surrogate model), SHAP (general model-agnostic version), occlusion or perturbation methods (black-box). [Springer Link](#)

Key implication: Model-specific methods often yield higher fidelity (explanation more truly aligned with model) but less generality; model-agnostic give broader applicability but might approximate or distort the actual reasoning.

4. Representation (the form/format of the explanation)

Definition: This dimension concerns how the explanation is presented to users (visual, textual, numeric, rule-based, etc.).

Some common representation types:

- **Feature attributions / importance scores:** Numeric values (or bars) showing how much each feature contributed to a decision (global or local).
 - *Example:* SHAP values showing positive or negative contribution of each feature to prediction.
- **Surrogate models / rule-based explanations:** A simpler interpretable model (e.g., decision tree or rules) approximates the complex model in a local or global region.
 - *Example:* LIME fits a linear model locally. Or a rule extraction method gives “IF income <30k AND credit_score <600 THEN risk=high”.
- **Visualizations / saliency maps** (especially in image / deep learning contexts): Heatmaps on input (e.g., highlighting parts of an image that influenced classification) or layer-activation visualisations.

- *Example:* Grad-CAM map showing which part of the image the CNN used to classify “cat”.

AryaXAI+1

- **Textual / natural language explanations:** Explanations written in human language, often summarising reasoning.
 - *Example:* “The model predicted loan rejection because the applicant’s debt-to-income ratio is above 40 %, and they have no prior credit history.”
- **Counterfactual or example-based explanations:** Showing “if X had been different then the result would change”.
 - *Example:* “If credit_score were 650 instead of 580, the model would have approved the loan.”
- **Graph- or flow-based explanations:** For models with complex dependencies, you might get explanation graphs or flows (less common in basic courses, but emerging). [Springer Link+1](#)

Key implication: Choose representation according to audience and use-case. A non-technical domain user might prefer textual explanations. A data scientist may prefer numeric attributions or visual heatmaps. Visualization works well for image models; feature attributions better for tabular data.

Summary Table

Dimension	Categories	Key distinguishing factor	Example(s)
Timing	Intrinsic (ante-hoc) / Post-hoc	Whether explanation is built into model or after	Decision tree (intrinsic); LIME on DNN (post)
Scope	Global / Local	Whether whole model behaviour or single prediction	Global SHAP summary; Local LIME per-instance
Model dependency	Model-specific / Model-agnostic	Whether uses model internals or treats black box	Grad-CAM (model-specific); LIME (agnostic)
Representation	Attributions / Surrogates / Visualization / Text / Counterfactual	Format of explanation	SHAP scores; rule-extraction; heatmap; narrative

Some concrete method-to-classification mappings

- **LIME:** Post-hoc, local, model-agnostic, representation → surrogate linear model (feature attributions + rule summary).
- **SHAP (general version):** Post-hoc, can be local or global (you can aggregate), model-agnostic, representation → feature attribution (numeric importance) and visual plots.
- **TreeSHAP** (for tree ensembles): Post-hoc, local/global, model-specific (for trees), representation → numeric attributions.
- **Grad-CAM:** Post-hoc, local (typically individual image), model-specific (CNNs), representation → visual heatmap.
- **Simple decision tree:** Intrinsic, global (model itself), model-specific (the tree is inherently interpretable), representation → rule-based or tree structure that can be read by humans.

- **Counterfactual explanation methods:** Often post-hoc, local, model-agnostic (in many cases), representation → “if-then” or example-based explanation.
-

Additional remarks (for completeness)

- The literature notes that while many taxonomies of XAI exist, no single universally accepted classification framework has emerged. [MDPI+1](#)
- The choice of XAI method must integrate use-case constraints: regulatory requirements, user competence, model complexity, data type, etc.
- Beware of trade-offs: interpretability vs fidelity vs generality. For example, a surrogate model might simplify behaviour but distort actual model logic. (Holzinger et al. note this tension) [Springer Link](#)
- When features are highly correlated (collinear), some attribution methods (especially model-agnostic ones) may give misleading importance scores; recent research emphasises this limitation.

[arXiv](#)

If you like, I can **append a diagram** or **mind-map** of the classification for your assignment notes, and also provide a list of **common methods mapped to each category**. Would you like that?



2. Outline steps in LIME for explaining a black-box model prediction.

Here is a detailed step-by-step outline of how LIME (Local Interpretable Model-Agnostic Explanations) works to explain a single prediction of a black-box model. This is structured in a way suitable for a 6-mark exam answer.

Steps in LIME

1. Select the instance to explain

- Choose the specific input instance x whose prediction $\hat{y} = f(x)$ from the complex model f you want to interpret. [DataCamp+3](#)
- Record the prediction value and class (for classification) or output (for regression).

2. Generate perturbed samples around the instance

- Create a set of synthetic or perturbed instances $\{x'_i\}$ by varying the features of x (for tabular: by sampling values for features; for text: by removing or masking words; for image: by toggling super-pixels). [christophm.github.io+2](#)
- For each perturbed instance x'_i , obtain the prediction $\hat{y}'_i = f(x'_i)$ from the black-box model.

[Medium](#)

3. Weight the perturbed samples by proximity (locality)

- Compute a proximity or “distance” measure between each perturbed instance x'_i and the original instance x .
- Apply a kernel (commonly exponential or RBF) to assign a weight $\pi_x(x'_i)$ that gives higher weight to samples very close to x . [christophm.github.io+1](#)
- This ensures the surrogate model focuses on the local region around x .

4. Train an interpretable (surrogate) model on the weighted dataset

- Use the dataset $\{(x'_i, \hat{y}'_i)\}$ with weights $\pi_x(x'_i)$ to train a simple, human-interpretable model g (e.g., linear regression, sparse linear model, decision tree). [christophm.github.io+1](#)
- The objective is to minimise a loss term $L(f, g, \pi_x)$ measuring how well g approximates f locally, plus a complexity penalty $\Omega(g)$ for interpretability. [christophm.github.io](#)
- Optionally restrict g to use only a small number of features (feature selection) so the explanation remains simple.

5. Extract and present the explanation from the surrogate model

- Interpret the coefficients (or rules) of g as explanations of how each feature contributes to the black-box model’s prediction for x . For example: “Feature A contributed +0.15 to the probability, Feature B contributed -0.05...”.
- Output a human-readable explanation (feature weights, or rule set, or visual) for the original instance.

- Because g is locally faithful, the explanation is valid in the neighbourhood of x , but not globally.

[FreeCodeCamp+1](#)

6. (Optional) Validate or inspect explanation fidelity and robustness

- Check how well the surrogate model's predictions match the black-box model's predictions in that local region (local fidelity). [christophm.github.io](#)
- Be aware of possible issues: sensitivity to sampling strategy, instability of explanation if neighbourhood is defined poorly, or if the black-box model is highly non-linear locally. [DataCamp+1](#)

Summary of workflow

- Identify the instance → perturb around it → get black-box labels → weight perturbations by proximity → fit simple surrogate → interpret surrogate.
 - Key design choices: how to perturb, how to measure proximity, which surrogate model, how many features.
 - The method is **local** (explains one instance) and **model-agnostic** (works for any black-box). [DataCamp](#)
-

If you like, I can **provide a diagram** of LIME's workflow and also include a **pseudo-code snippet** (in Java or Python) to demonstrate how LIME could be implemented. Would you like that?



3. Illustrate LIME application in email spam classification, showing perturbation and surrogate model.

Below is a concrete, exam-ready illustration of using LIME to explain a **single spam prediction**. I give a worked example, a small perturbation table, the surrogate model interpretation, runnable pseudo-code, and brief caveats.

Illustration summary.

We have a black-box spam classifier f . Input is one email:

“Congratulations! You win a free iPhone. Click this link to claim your account prize.”

The classifier predicts **spam probability = 0.87**. We use LIME to explain *why* f gave that prediction.

Step-by-step LIME application (text case)

1. Select instance

- Original text x above. Prediction $f(x) = 0.87$.

2. Define interpretable representation

- Choose tokens (words or superword-features) as interpretable features.
- For this example use binary presence of: **free, win, click, link, account, prize, congratulations**.

3. Generate perturbations

- Create many perturbed versions by randomly removing some tokens (masking). Each perturbed sample yields a binary vector indicating which tokens remain.
- Query the black-box for each perturbed text to get $f(x'_i)$.

4. Compute proximity weights

- Use a similarity kernel between original and perturbed text. For binary masks use similarity = fraction of tokens preserved or an exponential kernel on Hamming distance. Closer perturbations get higher weight.

5. Fit interpretable surrogate

- Fit a weighted sparse linear model g (e.g., LASSO or weighted linear regression) on dataset $(z_i, f(x'_i))$ where z_i are binary token vectors and weights are proximities.
- Extract coefficients as local feature attributions.

6. Report explanation

- Present top positive/negative contributors and optionally a counterfactual (which token removal would flip prediction).

Small concrete perturbation table (illustrative numbers)

Perturbations are binary for tokens [free, win, click, link, account, prize, congratulations]. Predictions are outputs from black-box f . Weights are proximity to original.

ID	Perturbed tokens present	Binary vector	$f(x'_i)$ (pred)	weight
0 (orig)	all tokens	1,1,1,1,1,1,1	0.87	1.00
1	remove free	0,1,1,1,1,1,1	0.62	0.9
2	remove win	1,0,1,1,1,1,1	0.70	0.9
3	remove click	1,1,0,1,1,1,1	0.68	0.9
4	keep only free,win	1,1,0,0,0,0,0	0.78	0.6
5	keep only account	0,0,0,0,1,0,0	0.18	0.4
6	remove all promo words	0,0,0,0,1,0,0	0.12	0.3
7	keep free,click	1,0,1,0,0,0,0	0.73	0.6

These prediction numbers are **illustrative** (hypothetical) to show how LIME uses model outputs. In a real run you would query the actual black-box model.

Fit surrogate and interpret coefficients

Fit weighted linear model $g(z) = \beta_0 + \sum_j \beta_j z_j$ using the table above and weights.

Example (hypothetical) fitted coefficients:

- Intercept $\beta_0 = 0.10$
- **free**: $\beta_{\text{free}} = +0.35$
- **win**: $\beta_{\text{win}} = +0.25$
- **click**: $\beta_{\text{click}} = +0.22$
- **link**: $\beta_{\text{link}} = +0.05$
- **account**: $\beta_{\text{account}} = -0.05$
- **prize**: $\beta_{\text{prize}} = +0.08$
- **congratulations**: $\beta_{\text{congratulations}} = +0.02$

Local explanation (readable):

- The words **free**, **win**, and **click** are the largest positive contributors to the spam score.
- **account** slightly reduces spam score in this local surrogate.
- Surrogate predicts approx $0.10 + 0.35 + 0.25 + 0.22 + 0.05 - 0.05 + 0.08 + 0.02 \approx 0.87$, matching the black-box locally.

Counterfactual insight: removing **free** drops surrogate score by ~ 0.35 to ~ 0.52 . Removing both **free** and **win** would drop it further. This gives a human-useful statement: *"The classifier labels this email as*

spam mainly because of words 'free', 'win', and 'click'. If 'free' were absent, spam probability would fall substantially."

Minimal Python pseudo-code (how you would implement LIME steps)

```
python

# PSEUDO-CODE (not a package call). For real use, see lime.lime_text.
from sklearn.linear_model import Ridge, Lasso
import numpy as np

orig_text = "Congratulations! You win a free iPhone. Click this link to claim your account prize."
tokens = ["free", "win", "click", "link", "account", "prize", "congratulations"]

def perturb_text(orig, tokens, num_samples=500, keep_prob=0.5):
    samples = []
    masks = []
    for _ in range(num_samples):
        mask = (np.random.rand(len(tokens)) < keep_prob).astype(int)
        # build perturbed text by keeping tokens with mask=1; keep other words too if desired
        pert_text = " ".join([t for t,m in zip(tokens,mask) if m==1])
        samples.append(pert_text)
        masks.append(mask)
    return samples, np.array(masks)

# 1. generate perturbed texts and mask matrix
texts, Z = perturb_text(orig_text, tokens, num_samples=200, keep_prob=0.7)

# 2. query black-box model f for each text -> get predictions y
y = [black_box_predict(p) for p in texts] # user-supplied model

# 3. compute proximity weights (e.g., exponential kernel on Hamming distance)
def kernel(original_mask, mask):
    dist = np.sum(np.abs(original_mask - mask))
    return np.exp(-(dist**2) / (sigma**2)) # choose sigma
orig_mask = np.ones(len(tokens)) # original has all tokens
weights = np.array([kernel(orig_mask, m) for m in Z])

# 4. fit sparse linear surrogate g using weights
model = Lasso(alpha=0.01) # sparse linear model
model.fit(Z, y, sample_weight=weights)
coefs = model.coef_
intercept = model.intercept_

# coefs now give local attributions per token
```

For production or coursework use, reference the `lime` Python package and its `LimeTextExplainer` which does these steps with text-specific perturbations and smarter tokenization.

Short limitations and best practices

- **Local fidelity vs global validity.** LIME explains neighbourhood behaviour only. Surrogate may be misleading outside that neighborhood.
- **Perturbation strategy matters.** Random masking can produce out-of-distribution inputs. For text, semantic perturbations or in-vocabulary replacements are safer.

- **Stability.** LIME explanations can vary with random seed, perturbation count, and kernel width. Run multiple times or use aggregated explanations for robustness.
 - **Correlated features.** When tokens/features are collinear, coefficient attribution can be unstable.
-

If you want, I will:

- Run a **real LIME example** on a specific spam classifier you provide or code a toy classifier and show actual outputs.
- Produce the same explanation in the **format your assignment needs** (short answer, bullet points, or diagram).



4. Discuss advantages and limitations of LIME, including flexibility and computational cost.

Here are the major **advantages** and **limitations** of LIME (Local Interpretable Model-Agnostic Explanations), with a focus on flexibility, computational cost, fidelity and practical trade-offs. Use these for the 6-mark style answer.

Advantages

1. Model-agnostic flexibility

- LIME works with virtually any model (tree, neural network, ensemble, even black-box) because it treats the underlying model as a “black box” and doesn’t require internal access. [C3 AI+1](#)
- It supports different data types (tabular, text, image) by constructing interpretable representations (words, super-pixels etc). [christophm.github.io+1](#)
- Because of this flexibility, practitioners can use LIME across diverse domains without redesigning separate explanation methods for each model type.

2. Local interpretability

- It provides an explanation targeted at a single instance (local scope). That is highly useful when you need to justify or understand a particular decision rather than overall model behaviour. [Analytics Vidhya+1](#)
- This helps in domains with high stakes (e.g., medicine, credit) where you need to explain *why this decision* occurred for *this case*.

3. Relatively straightforward implementation and comprehension

- LIME’s process (perturb → weigh → surrogate) is conceptually simple and the surrogate model (e.g., sparse linear model) is interpretable by humans. [christophm.github.io+1](#)
- Many libraries support LIME (Python, R) so adoption is easy in practice.

4. Interpretable feature space decoupled from model representations

- Even if the model is trained on complex or transformed features (e.g. embeddings, PCA), LIME lets you define interpretable features (e.g. words, original attributes) for explanation, boosting human-readability. [christophm.github.io](#)

Limitations

1. Local scope only—no global model explanation

- Because LIME explains one instance at a time, you don’t get a full picture of the model’s overall behaviour, feature interactions, or global biases. [Medium+1](#)
- If you try to infer global insights from many LIME outputs you risk misinterpretation.

2. Fidelity and faithfulness concerns

- The surrogate model approximates the black-box only in a local region; the fidelity of the explanation depends on how well it approximates. If the model is highly non-linear in that region or perturbation does not reflect realistic inputs, the explanation may be misleading.

slds-lmu.github.io

- Research shows that slight changes in the input or hyper-parameters can produce quite different explanations, indicating instability. [Medium+1](#)

3. Stability / reproducibility issues

- Because LIME uses randomness in the perturbation/sampling process and user-tunable parameters (kernel width, number of samples, neighborhood definition) the explanation for the same instance can vary across runs. christophm.github.io+1
- That undermines trust if one expects reliable, repeatable explanations.

4. Computational cost and inefficiency for many instances

- For each instance you must generate many perturbed samples, query the model many times, compute weights, train a surrogate model → this can be computationally expensive especially if the underlying model is large or the dataset is high-dimensional. [Medium+1](#)
- If you need explanations for many predictions (batch explanations), cost multiplies and might become impractical. [Medium](#)

5. Perturbation realism and feature correlation issues

- The perturbation strategy often ignores feature correlations; generating unrealistic samples can cause the surrogate to learn behaviour that is not representative of real data. christophm.github.io+1
- In domains like images the standard perturbation (painting super-pixels black) may confuse the model and cause explanations to reflect artifact rather than actual feature relevance. [Springer Link](#)

6. Hyper-parameter sensitivity

- Choices such as kernel width (which defines “local”), number of samples, surrogate model complexity, sample weighting all influence the output explanation and must be carefully tuned. Yet there is no universal “best” setting. slds-lmu.github.io+1

Summary of trade-offs & guidance

- Use LIME when you need **instance-level explanations**, have a black-box model, and you can afford per-instance cost.
- Do *not* rely on LIME for global model auditing unless you combine many local explanations and interpret them cautiously.
- Ensure you sample enough perturbations, check surrogate fidelity (how well it fits the black-box locally), and verify stability (repeat runs).
- Be aware of domain specifics: perturbation must keep samples realistic; feature correlation must be considered; for high dimensional or very non-linear regions, explanations may degrade.
- If you need explanations for many instances in production or require global consistency and stability, consider complementing LIME with other methods (e.g., SHAP) or global interpretable models.

If you like, I can **provide a table** summarising each advantage vs limitation with references and include **recent research findings** (2023-24) on LIME's stability and computational improvements—would that be helpful?



5. Explain SHAP using game theory, with Shapley value calculation for feature contributions

Here is a detailed, exam-ready explanation of **SHAP (SHapley Additive exPlanations)**: how it uses the game-theoretic concept of the Shapley value, what the calculation involves, and how that maps to feature contributions in machine-learning models.

Game-theoretic foundation: Shapley value

- In cooperative game theory we imagine a set of *players* $N = \{1, 2, \dots, n\}$. A **value** (or payoff) function $v(S)$ assigns a real number to any coalition $S \subseteq N$. [Wikipedia+3](#)
- The Shapley value $\varphi_i(v)$ for player i is defined as the average *marginal contribution* of i over all possible orders in which players could join the coalition. One formula:

$$\varphi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} (v(S \cup \{i\}) - v(S))$$

This computes for each subset S not containing i the marginal contribution of adding i , weighted by how many permutations produce that subset. [Wikipedia+1](#)

- The Shapley value satisfies key fairness axioms: efficiency (sum of values equals total value of grand coalition), symmetry, dummy (zero value if contribution always zero), additivity. [Medium+1](#)
- In ML interpretability the “players” become features, the “coalitions” become sets of features present, and $v(S)$ becomes something like “model output given features in S ” (or expected model output when only S are present) in order to attribute credit (or blame) to features.

Mapping to SHAP for model explanations

- We consider a predictive model $f: \mathbb{R}^n \rightarrow \mathbb{R}$ (for classification or regression). We pick an instance x whose prediction we want to explain.
- Define a reference (background) value $E[f(X)]$ or other baseline (e.g., average model output over background dataset). SHAP attributes satisfy

$$f(x) = E[f(X)] + \sum_{i=1}^n \phi_i$$

where ϕ_i is the SHAP value for feature i for instance x . [GitHub+2](#)

- In effect we define a cooperative game: features are players; each subset S of features has a value

$$v(S) = E[f(x) \mid \text{features in } S \text{ fixed to their value in } x, \text{ others marginalized}]$$

Then the Shapley value formula gives each feature i a contribution ϕ_i . The sum of all ϕ_i plus the baseline equals the model’s output. [SHAP+1](#)

Shapley value calculation for feature contributions: a worked mini-example

Suppose you have a model f with **two** features: Feature A and Feature B. For a given instance x , the model prediction is $f(x) = 0.8$. The baseline (expected) model output is $E[f] = 0.5$.

Define value function v as:

- $v(\emptyset) = E[f] = 0.5$
- $v(\{A\}) = 0.65$ when only A's value is known/fixed and B is marginalized
- $v(\{B\}) = 0.60$ when only B's value is fixed
- $v(\{A, B\}) = f(x) = 0.8$ when both features fixed

Compute contributions:

- For Feature A:

$$\varphi_A = \frac{1}{2!} \left[(v(\{A\}) - v(\emptyset)) + (v(\{A, B\}) - v(\{B\})) \right] = \frac{1}{2} \left[(0.65 - 0.5) + (0.8 - 0.6) \right] = \frac{1}{2} \times ($$

- For Feature B:

$$\varphi_B = \frac{1}{2!} \left[(v(\{B\}) - v(\emptyset)) + (v(\{A, B\}) - v(\{A\})) \right] = \frac{1}{2} \left[(0.60 - 0.5) + (0.8 - 0.65) \right] = \frac{1}{2} \times ($$

Check: Baseline + contributions = $0.5 + 0.175 + 0.125 = 0.8 = f(x)$. Good. So for the instance, Feature A contributed 0.175 above baseline; Feature B contributed 0.125.

In SHAP parlance we say "Feature A has SHAP value +0.175, Feature B has +0.125" for this instance.

Why this matters & how SHAP uses it

- SHAP extends this concept to **multiple features**, high-dimensional models, and can compute both local (instance-level) and global (aggregated) explanations (by aggregating $|\phi_i|$ over instances).
- SHAP builds on the axiomatically justified nature of the Shapley value: fair attribution of contributions. [Christoph M+1](#)
- In practice SHAP uses efficient approximations (like TreeSHAP for tree-models) to avoid full enumeration of all 2^n feature subsets. [SHAP+1](#)

Key points for your assignment

- The game-theoretic "players = features, game value = model output difference from baseline".
- The formula for Shapley value and how you compute marginal contributions for each feature.
- The additive explanation property: model output = baseline + sum of SHAP values.
- The fairness axioms (efficiency, symmetry, dummy, additivity) justify why Shapley values are appealing for XAI.
- Highlight that although conceptually simple, computational cost can be high (exponential in features) so approximations are needed.

If you like, I can prepare **pseudo-code** (in Java or Python) for computing SHAP values for a small model, plus show how you'd interpret the output in tabular form for your exam answer.



6. Describe mathematical representation of SHAP, including baseline and feature values.

Mathematical Representation of SHAP

SHAP (SHapley Additive exPlanations) expresses a model's prediction as a **sum of baseline output plus feature contributions** derived from game theory.

1. Additive Explanation Model

The general additive form of SHAP is:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

Where:

- $f(x)$ = model output for instance x .
- ϕ_0 = **baseline value** (expected prediction when no feature information is known).
- ϕ_i = **SHAP value** (contribution of feature i to deviation from the baseline).
- M = number of input features.

Thus,

$$\text{Prediction} = \text{Baseline} + \text{Sum of feature effects.}$$

2. Baseline Value (ϕ_0)

The baseline is typically the **expected model output** across the training or background dataset:

$$\phi_0 = E_X[f(X)]$$

This represents the model's average prediction before seeing any feature values.

Example:

If a model predicts probability of loan approval, and the average approval rate in the data is 0.60, then $\phi_0 = 0.60$.

3. Feature Contributions (ϕ_i)

Each SHAP value ϕ_i quantifies how much feature i moves the prediction from the baseline.

Derived from Shapley values in cooperative game theory:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{i\}) - v(S)]$$

Where:

- $N = \{1, 2, \dots, M\}$ = set of all features.
- S = subset of features excluding i .
- $v(S)$ = model's expected prediction using only features in S .

The weight term

$$\frac{|S|!(M - |S| - 1)!}{M!}$$

ensures **fair averaging** over all possible feature orderings.

4. Value Function $v(S)$

The coalition value function is:

$$v(S) = E[f(X) \mid X_S = x_S]$$

Meaning: expected model output given that the features in subset S take their actual values from instance x , while the remaining features are marginalized (averaged out).

Intuitively, $v(S)$ measures what the model predicts if it "knows" only features in S .

5. Complete SHAP Equation

Combining baseline and all feature contributions:

$$f(x) = E[f(X)] + \sum_{i=1}^M \phi_i$$

Thus SHAP decomposes the prediction into **baseline + fair share of each feature's marginal effect**.

6. Example

For a 3-feature model predicting credit approval:

$$E[f(X)] = 0.50, \quad \phi_1 = +0.20, \quad \phi_2 = -0.05, \quad \phi_3 = +0.10$$

Then:

$$f(x) = 0.50 + (0.20 - 0.05 + 0.10) = 0.75$$

Meaning the model's prediction (0.75 approval probability) is 0.25 higher than baseline due to specific feature effects.

7. Properties (Axioms Satisfied)

SHAP values uniquely satisfy:

1. **Efficiency:** $\sum_i \phi_i = f(x) - E[f(X)]$
2. **Symmetry:** Equal contributors receive equal value.
3. **Dummy:** Features that do not affect the prediction get $\phi_i = 0$.
4. **Additivity:** Explanations are additive over models.

8. Summary Table

Symbol	Meaning	Description
$f(x)$	Model prediction	Output for instance x
$E[f(X)]$	Baseline	Expected output without features
ϕ_i	Feature contribution	Marginal effect of feature i
$v(S)$	Value function	Expected output using subset S
M	Number of features	Total count of "players"
Formula	$(\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{v(S \cup \{i\}) - v(S)}{ S })$	S

9. Interpretation

- $E[f(X)]$: model's "neutral" prediction.
- $\phi_i > 0$: feature pushes prediction *upward* relative to baseline.
- $\phi_i < 0$: feature pushes prediction *downward*.
- Sum of all ϕ_i : total deviation from baseline.

Would you like me to follow this with **Question 7 (BETA surrogate models in credit approval)** next?



7. Apply SHAP to a loan repayment prediction, interpreting feature impacts

Local and global SHAP applied to a loan-repayment prediction, with concrete numbers, interpretation, and a short implementation recipe.

Setup and goal

Predictor f outputs **probability of default** for a borrower.

Baseline (expected) default risk $\phi_0 = E[f(X)] = 0.15$ (15%).

We explain a single borrower x for whom $f(x) = 0.46$ (46% default probability). SHAP decomposes:

$$f(x) = \phi_0 + \sum_i \phi_i \Rightarrow 0.46 = 0.15 + \sum_i \phi_i.$$

Hypothetical features and SHAP values (local explanation)

Feature	Description	SHAP value ϕ_i	Effect on prediction
credit_score	FICO score (higher better)	-0.18	Reduces default risk by 18 percentage points
income	Monthly income (₹)	-0.06	Reduces risk by 6 pp
loan_amount	Requested loan size	+0.10	Increases risk by 10 pp
dti	Debt-to-income ratio	+0.12	Increases risk by 12 pp
employment_length	Years employed	-0.02	Slightly reduces risk by 2 pp
prior_delinquencies	Number past delinquencies	+0.15	Increases risk by 15 pp
age	Borrower age	+0.00	No material effect locally

Check sum: $0.15 + (-0.18) + (-0.06) + 0.10 + 0.12 + (-0.02) + 0.15 + 0.00 = 0.46$.

Local interpretation (single borrower)

- Largest positive contributors: **prior_delinquencies** (+0.15) and **dti** (+0.12). These push predicted default upward.
- Largest negative contributor: **credit_score** (-0.18) — a strong protective factor.
- Net effect: borrower's good credit score and income partially offset high DTI, loan size and past delinquencies, producing 46% default probability.

Human-readable statement:

"This borrower's baseline risk is 15%. Past delinquencies and a high debt-to-income ratio raise risk substantially. A strong credit score reduces risk most, but not enough to offset the adverse factors. The net predicted default probability is 46%."

Global view (aggregate uses)

Compute mean absolute SHAP values across many instances to rank global feature importance:

Feature	Mean $ \phi $ (example)	Global rank
---	---	---
credit_score	0.12	1
prior_delinquencies	0.09	2
dti	0.08	3
loan_amount	0.05	4
income	0.04	5

Use: regulators or product managers use this to see which features drive model behavior overall. Local explanations still required for individual decisions.

Interactions and counterfactuals

- SHAP interaction values can show feature pairs whose combined effect differs from the sum of marginals (e.g., `loan_amount` $\times$ `income`).
- Counterfactual statements derived from SHAP are plausible: e.g., "If `prior_delinquencies` were 0 instead of 2, predicted default would drop by ~15 pp." (careful: depends on how $v(S)$ is computed and correlated features).

Short Python recipe (Tree model, production style)

```
python

import shap
# model: trained tree-based model (XGBoost/LightGBM/RandomForest)
# X_train: background dataset (used to compute baseline); X_row: single instance to explain

explainer = shap.TreeExplainer(model, data=X_train)           # TreeSHAP is exact for tree models
shap_values = explainer.shap_values(X_row)                   # returns array of phi_i for X_row
base_value = explainer.expected_value                        # baseline phi_0

# display local force plot (Jupyter)
shap.force_plot(base_value, shap_values, X_row)

# global importance (mean absolute)
shap.summary_plot(shap_values_all, X_all, plot_type="bar")
```

Notes: for classification with two classes `shap_values` may return a list; use the class-specific values. Use a representative background set for `data=` to get meaningful baselines.

Practical cautions and best practices

- **Baseline choice matters.** $E[f(X)]$ depends on the background dataset. Use a representative sample.
- **Feature correlation.** When features are correlated, marginalizing others can produce misleading attributions. Interpret cautiously.

- **Computational cost.** Exact Shapley enumeration is exponential. Use TreeSHAP (exact for trees), KernelSHAP (approximate, model-agnostic), or sampling approximations.
 - **Stability.** Use consistent background data and verify explanations on multiple nearby instances. Aggregate local explanations to validate global claims.
 - **Regulatory use.** For compliance, combine SHAP with model documentation and human review. Don't present SHAP as the sole justification.
-

One-line summary

SHAP gives a fair, additive decomposition of the prediction into baseline + feature contributions. For the loan case it identifies which borrower attributes increased or decreased default risk and by how much, enabling transparent, auditable decisions when used with care.



8. Explain Diverse Counterfactual Explanations (DiCE) for generating alternative outcomes.

Here's a detailed, exam-ready explanation of **DiCE (Diverse Counterfactual Explanations)**—what it is, how it works, and why it matters.

What is DiCE?

DiCE is a framework and software library (Python: `dice-ml`) for generating multiple “what-if” alternative instances (counterfactuals) for a given prediction from a machine learning model. [InterpretML+2](#)

Rather than just explaining *why* a model gave a certain output (e.g., via feature attributions), DiCE shows *how* the input could be changed so the output would change (e.g., “You would have been approved if your income were higher by ₹10,000”). [InterpretML+1](#)

Crucially DiCE emphasizes **diversity** of the counterfactuals (multiple plausible ways to change the outcome) and **feasibility** (the changes should be actionable and realistic). [arXiv+1](#)

Key attributes / criteria of good counterfactuals in DiCE

Based on literature and the DiCE implementation: [Christoph M+2](#)

- **Validity:** The counterfactual instance x' should result in the *desired* changed output (e.g., model flips class or crosses a threshold).
- **Proximity / similarity:** x' should be as close as possible to the original instance x (minimal changes).
- **Sparsity:** Change as few features as possible.
- **Plausibility / feasibility:** The changed features should be realistic, feasible for that individual, and within permitted ranges.
- **Diversity:** Generate *multiple* counterfactuals that are different from each other in meaningful ways (so choices can be compared).
- **Actionability:** Feature changes should ideally be to those features that can be acted on (not immutable ones like age, gender) when the context demands.

How DiCE works (high-level algorithm)

1. Define the data, model, and query instance

- Dataset wrapping via DiCE's `Data` object (specifies continuous vs categorical features, permitted ranges). [InterpretML+1](#)
- The trained model (black-box or white-box) wrapped via DiCE's `Model` object.
- A query input instance x whose model output you want to alter.

2. Specify desired outcome and constraints

- For classification: desired class or for regression: target value or threshold.
- Constraints: which features can vary (**features_to_vary**), permissible ranges or categories (**permitted_range**), which features are immutable. [InterpretML+1](#)

3. Formulate optimization / search for counterfactuals

- DiCE treats the counterfactual generation as an optimisation or search problem: find x' that changes model output to desired state, while minimising distance to original and meeting sparsity, diversity, and feasibility. [arXiv+1](#)
- Methods used: randomised search, genetic algorithms, KD-tree nearest neighbor search for tabular, gradient-based search for differentiable models. [Microsoft Learn+1](#)

4. Generate a set of counterfactuals $\{x'_1, x'_2, \dots, x'_k\}$

- Ensures diversity: the solutions differ in which features are changed and by how much.
- Evaluate each candidate: check whether it achieves desired output, compute distance, check feasibility.

5. Return explanations

- Provide the user multiple actionable scenarios: e.g., "If you increase your income to ₹50,000 and reduce your debt to income ratio to 30 % you would have been approved."
- I.e., each x'_j is paired with changed feature values and the alternative model outcome.
- Optionally compute local feature importance via frequency of feature changes across counterfactuals. [InterpretML](#)

Example scenario (loan approval)

You have a model that denies a loan for applicant x . DiCE might output 3 counterfactuals:

- CF1: Increase monthly income by ₹12,000 → loan approved.
- CF2: Reduce number of existing loans from 3 to 1 → approved.
- CF3: Extend employment length from 1 year to 3 years and reduce DTI from 45% to 35% → approved.

These are different paths (diverse) the applicant could take. They are actionable and feasible (income raise, pay off loans, longer employment). By comparing them the applicant gets choice.

Why DiCE matters

- For **decision-subjects** (people affected): gives actionable recourse (what changes you should make to get a favourable outcome).
- For **model auditors** or fairness checks: by generating many counterfactuals you can see whether the model behaves unfairly or has brittle decision boundaries. [Microsoft+1](#)
- Complements attribution methods (like SHAP/LIME) by moving from *why* to *how to change*.

Limitations & caveats

- Feasibility is complex: even if a counterfactual says “increase income by ₹12,000”, that may not be realistic for the individual. DiCE allows constraints but full causal/actionable realism is hard.
[InterpretML+1](#)
- Proximity/diversity trade-off: You might generate counterfactuals which are too far from the original or too many changes.
- For high-dimensional or deep models the search can be expensive.
- Counterfactuals are not guarantees: they show what *could* change the outcome *under the model*, not that these changes will work in real life (no guarantee of causal effect).
- Diversity means not all counterfactuals are equally meaningful; the user must interpret and choose which path is feasible for them.
- Immutability and domain constraints: Changing immutable features (age, gender) is unhelpful or biased. The tooling must handle these.

Summary

- DiCE produces **diverse counterfactual explanations** by searching for input modifications that flip a model’s decision.
- It emphasises **diversity, feasibility, actionability**, and **proximity**.
- Works model-agnostic, applicable in high-stakes domains like finance, healthcare, etc.
- Use it to provide meaningful “what-if” scenarios rather than just feature importance.
- But you must check feasibility, user context, and understand it reflects model behaviour not necessarily real-world causality.

If you like, I can **provide pseudo-code** for DiCE usage (Python snippet) and **an assignment-style formatted answer** (bulleted + glossary) you can use verbatim in your submission.



9. Discuss Layer-wise Relevance Propagation (LRP) for attributing predictions in neural networks.

Layer-wise Relevance Propagation (LRP)

 The image has expired. Refresh ChatGPT page and retry saving to PDF.

 The image has expired. Refresh ChatGPT page and retry saving to PDF.

 The image has expired. Refresh ChatGPT page and retry saving to PDF.

Description

Layer-wise Relevance Propagation (LRP) is a model-specific interpretability method designed for neural networks. It works by **redistributing** the model's output prediction backward through the network layers to assign *relevance scores* to individual input features (e.g., pixels, words). [IPHome+2](#)

Key characteristics:

- It relies on the internal structure of the neural network (weights, activations) → hence *model-specific*.
- It uses rules for propagating relevance from a higher layer neuron to lower layer neurons, preserving a **conservation property**: total relevance at each layer equals the relevance at the next layer. [Fraunhofer Heinrich-Hertz-Institut+1](#)
- The end result is often a heatmap over the input indicating which features “supported” (positive relevance) or “opposed” (negative relevance) the output. [PMC+1](#)

How it works (in exam-style steps)

1. **Forward pass**: Input x is passed through the network; the model produces output $f(x)$ (for example, a class score or probability).
2. **Initialize relevance at output**: Set relevance of the output neuron(s) equal to the output score (or class logit) you want to explain. Denote this as $R_k^{(L)}$, where L is the final layer, k is the output neuron of interest. [IPHome](#)
3. **Backward propagation of relevance**: For each layer l from L down to input layer (layer 0):
 - For each neuron j in layer l , distribute its relevance $R_j^{(l)}$ to neurons i in layer $l - 1$ according to a propagation rule:

$$R_i^{(l-1)} = \sum_j \frac{a_i w_{ij}}{\sum_i a_i w_{ij} + \epsilon \cdot \text{sign}(\sum_i a_i w_{ij})} R_j^{(l)}$$

where a_i is activation of neuron i , w_{ij} weights, ϵ small stabiliser. (One example rule.) [IPHome+1](#)

- Ensure that $\sum_i R_i^{(l-1)} = \sum_j R_j^{(l)}$. This is the relevance conservation.

4. **At input layer:** You obtain relevance scores $R_i^{(0)}$ for input features. These indicate how much each feature contributed to the decision.
5. **Interpretation:** Positive R_i means that feature pushed the model toward the decision; negative means it pushed away. Visualise via heatmap in image, or table in tabular data.

Propagation Rules

Different layers (dense, convolutional, pooling, normalisation) may require different propagation rules. Common variants:

- **z -rule:** $R_i = \sum_j \frac{z_{ij}}{\sum_i z_{ij}} R_j$, where $z_{ij} = a_i w_{ij}$.
- **ε -rule:** Adds small ε in denominator for numerical stability.
- **$\alpha\beta$ -rule:** Split positive and negative contributions with parameters α, β .
Selecting rules impacts explanation granularity and fidelity. [IPHome](#)

Advantages

- Aligns with internal model structure → potentially high fidelity explanations for neural networks.
- Generates detailed pixel/feature-level relevance for deep models (e.g., CNNs) which helps in domains like image classification or medical imaging. [Frontiers+1](#)
- Conservation property helps ensure explanation sums match output, making attributions additive and consistent.
- Works for single instances (“local”) and provides fine-grained insights: “This pixel/feature contributed this much”.

Limitations

- Model-specific: Only for neural networks (or architectures with known structure) not general black-box.
- Rule selection and implementation complexity: Choosing propagation rule parameters ($\varepsilon, \alpha, \beta$) is non-trivial and can significantly affect the results. [IPHome+1](#)
- Vulnerable to numerical instability especially with very deep or highly non-linear networks. Small denominators lead to large relevance fluctuations. [AiEnsured](#)
- Interpretability challenge: Relevance scores may still be hard for non-technical users; heatmaps may highlight regions but not always convey “why”.
- Not inherently causal: It attributes relevance based on network mechanics, not necessarily real-world causality; correlated features may get misleading relevance.
- For very large networks the backward propagation of relevance may still be computationally intense, especially when many layers and variants are considered.

Use-Case Example

In a CNN trained to detect disease in MRI scans (e.g., Alzheimer’s), LRP was used to generate heatmaps for each patient showing which brain voxels were most relevant for the network’s “disease” classification.

It was shown that in healthy controls there was very little relevance, and in patients relevance corresponded to known temporal-lobe regions. [PMC+1](#)

When to use LRP

- When model is a deep neural network and you can access its internal weights/activations.
- When you need detailed attributions at the input-feature (pixel, token, sensor) level.
- When you want the explanation to align with internal model structure rather than surrogate approximations.
- When the domain demands high fidelity explanations (e.g., medical imaging, safety-critical systems).

Summary

LRP is a powerful neural-network-specific explanation technique that propagates the prediction back through layers to assign relevance scores to inputs. It balances fidelity (thanks to using model internals) with interpretability (via heatmaps or feature scores). However, it requires careful configuration, is limited to models with accessible internals, and may impose computational and usability burdens.

If desired, I can provide a **mathematical derivation** of the deep-Taylor decomposition underlying LRP, and a **Python/PyTorch snippet** showing how to implement a simple LRP rule for a small feed-forward network.



Unit 4 Chapter 4.2: Trust and acceptance

1. List metrics to evaluate XAI, such as fidelity and robustness.

Here are key metrics used to evaluate XAI methods, particularly under the themes of **trust** and **acceptance**. I group them with definitions and brief notes.

Metrics to evaluate XAI

1. Fidelity (Faithfulness)

- Definition: How closely the explanation reflects the actual behaviour of the model (i.e., input→-output mapping) rather than some surrogate or approximation. [MDPI+3](#)
- Notes: High fidelity means the explanation accurately captures what the model did. Low fidelity undermines trust. [ScienceDirect+1](#)

2. Robustness / Stability

- Definition: How consistent the explanation is under small perturbations of the input or under minor changes to the model or data. [CVF Open Access+1](#)
- Notes: If small changes in input cause wildly different explanations, trust may drop.

3. Sparsity / Simplicity

- Definition: How concise or minimal the explanation is (e.g., number of features used, length of explanation). [PMC+1](#)
- Notes: Simpler explanations are easier for human comprehension and acceptance.

4. Comprehensibility / Interpretability

- Definition: How easily a human stakeholder can understand the explanation (form, language, visualization). [PMC+1](#)
- Notes: Even a high-fidelity explanation might be useless if users cannot interpret it.

5. Plausibility / Trustworthiness

- Definition: How convincing the explanation is to the human user, whether the explanation aligns with domain knowledge or user expectation. [PMC+1](#)
- Notes: This is subjective and bridges to user acceptance.

6. Generalisability

- Definition: The extent to which the explanation method works well across instances, data domains, or models (not just for one input). [PMC](#)
- Notes: Important if you want the XAI method to scale beyond a few cases.

7. Coverage

- Definition: Proportion of model decisions for which explanations can be generated. [MedRxiv](#)
- Notes: A method that explains only 10% of outputs might have limited utility.

8. Contrastivity / Discriminativeness

- Definition: The explanation's ability to show differences between model decisions or classes (i.e., what distinguishes this decision from another). [arXiv](#)
- Notes: Helps users see *why this* decision vs *that* decision.

9. Actionability / Recourse

- Definition: Whether the explanation enables a user to take meaningful action (e.g., change input features) to affect future decision.
- Notes: Especially important in high-stakes domains (credit, healthcare).

10. Computational Efficiency

- Definition: The time, resources, and cost required to generate the explanation.
- Notes: A method may be high fidelity but too slow for production.

11. Consistency Across Methods

- Definition: Whether different explanation methods produce similar explanations for the same model and input (or at least comparable).
- Notes: Helps build confidence in the explanation's validity.

If you like, I can **provide a table** with each metric, how to compute it (mathematical formulation or proxy), typical thresholds, plus recent research references.



2. Analyze trustworthy explainability acceptance in AI systems

Here is a detailed analysis of **trustworthy explainability and acceptance** in AI systems—why it matters, what factors drive it, and what to watch out for.

Why trust and acceptance matter

- Even high-accuracy AI models will fail to be adopted if end-users don't trust them. For example, opaque "black-box" systems generate resistance in high-stakes domains. [ericsson.com+1](#)
- The concept of "trustworthy AI" links explainability, transparency, reliability, fairness and user acceptance. [arXiv+1](#)
- Explainability is a *means* to trust/acceptance, not a guarantee. Without alignment between explanation, user expectations and model behavior, acceptance won't follow. [arXiv](#)

Key factors influencing explainability → acceptance

1. Explanation fidelity & accuracy

- Explanations must reflect what the model actually does ("fidelity") so that users don't feel misled. [arXiv+1](#)
- Users tend to trust systems more when both accuracy and fidelity are high, rather than explanation alone. [arXiv](#)

2. User understanding & context tailoring

- Different users (technical developers vs end-users) have different information needs: internal model workings vs result behavior. [arXiv](#)
- Explanations must be understandable (simplicity, appropriate representation) for the target user to accept them.

3. Transparency and disclosure of limitations

- Disclosure about model limitations, possible failures or uncertainties improves trust and acceptance. [arXiv](#)
- Transparency includes data provenance, decision logic, roles of human oversight.

4. Robustness, fairness & reliability

- Trust is undermined if the system behaves unexpectedly or unfairly—even if explanations are available. [arXiv](#)
- Explainability must go hand in hand with ethical design, data governance and robust model lifecycle.

5. Actionability and recourse

- Users are more likely to accept an AI system if the explanation gives them actionable insight (“what I can do”) rather than just “why”.
- This supports human-in-the-loop control and builds agency.

6. User-system interface and interaction

- The method of presenting explanations (visual, textual, interactive) influences acceptance. For example, interactive explanation tools or natural-language summaries boost usability. [arXiv](#)

7. Organisational and regulatory environment

- Acceptance is not just technical: organisational trust, regulatory compliance (e.g., right to explanation), user training matter. [ericsson.com+1](#)

Challenges and tensions

- **Trade-off between performance and interpretability:** Complex models (deep nets) may perform best but are hardest to explain. Acceptability may require balancing performance and transparency. [arXiv](#)
- **Over-trust vs under-trust:** Good explanations might lead users to over-trust systems (automation bias) or conversely detailed but poor explanations might reduce trust. Explaining doesn’t mean “trusted”. [Wikipedia](#)
- **Explanation fatigue or mismatch:** If explanations are too technical, irrelevant or inconsistent with domain knowledge, users may reject them.
- **Correlation $\neq$ causation:** Explanations may reflect model logic but not real-world causal mechanisms. Users may mistrust such if expectations differ.
- **Scalability:** Generating high-quality tailored explanations for each user and decision is operationally challenging.
- **Diverse user groups:** What builds trust for one stakeholder (e.g., regulator) might differ from what a front-line operator needs.

Practical guidelines to boost acceptance

- Design explanations with **user-centricity**: assess user role, expertise, goals, and tailor format and depth accordingly.
- Combine **technical validity** (fidelity, robustness) with **human usability** (simplicity, clarity).
- Embed **explainability by design** in the AI development lifecycle, not just add post-hoc. [ericsson.com](#)
- Provide **transparency of limitations**: e.g., “Model is less accurate for this subgroup” to manage expectations.
- Support **human oversight and recourse**: let users challenge or override model decisions and see explanations for that.
- Evaluate acceptance metrics: measure not only explanation metrics (fidelity etc) but also **user acceptance, trust levels, behavioural change**. E.g., the “Trustworthy Explainability Acceptance” metric proposed by Kaur et al. [SpringerLink+1](#)

- Organisationally: train users, build culture of AI literacy, monitor for unintended effects (bias, drift), and provide governance.
-

Summary

- Explainability is necessary but **not sufficient** for trust and acceptance in AI systems.
- Trustworthy explainable AI requires alignment across **model behavior, explanation quality, user understanding, actionability**, and **governance**.
- Without this alignment, AI systems risk being technically sound yet socially rejected.

If you like, I can supply **a checklist** of trust and acceptance criteria for your assignment (for a high-stakes domain like finance or healthcare) plus **recent empirical studies** (2023–24) on user acceptance of XAI systems.



3. Describe Power Quality Disturbance (PQD) classification using XAI.

Here is a structured, exam-ready description of how **Power Quality Disturbance (PQD) classification** can be implemented using XAI (Explainable Artificial Intelligence) methods. It covers: what PQDs are, how classification works, how XAI is integrated, and key considerations.

1. What are Power Quality Disturbances (PQDs)

- PQDs refer to deviations from normal voltage/current waveforms in power systems—such as sags (voltage drop), swells (voltage rise), interruptions, harmonics, flicker, transients, frequency deviations. [TalTech+3](#)
- These disturbances impact equipment reliability, performance and lifespan. [Brunel University Research Archive+1](#)
- The classification task: given a measured waveform (voltage/current time-series) determine which disturbance type (or combination) occurred.

2. Standard ML/AI classification workflow for PQDs

1. **Data acquisition:** capture voltage/current waveforms, typically many samples across disturbance events.
2. **Pre-processing / feature extraction:**
 - Time-frequency transforms (e.g., S-transform, wavelet transform) convert signal into features. [PMC+1](#)
 - Other features: RMS variation, harmonic content, skewness/kurtosis of decomposed modes. [arXiv](#)
3. **Model training:** Use a classifier (SVM, random forest, CNN, LSTM) to map features → disturbance class. Several recent works show very high accuracy ($\approx 97\text{-}99\%$). [MDPI+1](#)
4. **Deployment / inference:** When a disturbance occurs, the model predicts the class and triggers mitigation.

3. Role of XAI in PQD classification

- Highly accurate classifiers (especially deep models) are often **black-boxes**. In power systems, engineers and operators need *explanations* (why did the model decide this disturbance class?) to trust the system. [Technion CRIS+1](#)
- XAI methods help **interpret** model decisions: identify which signal features or time-frequency components led to the classification.
- Example: In the study “Explaining the decisions of power quality disturbance classifiers ...” the authors use a latent-space encoder/decoder to map back importance scores to original features for PQD classifiers. [Technion CRIS+1](#)

- Benefits: improved operator trust, easier fault diagnosis, traceable decisions, regulatory compliance.

4. How to integrate XAI into PQD classification – step-by-step

1. Train a high-performance disturbance classifier f .
2. Select an XAI technique appropriate to the model and domain: e.g., feature attribution (SHAP, LIME), saliency maps (for CNN on time-frequency images), latent-space importance.
3. For a given disturbance event sample x with predicted class $y = f(x)$:
 - Compute feature attributions: e.g., determine which time-frequency bands, which harmonic orders, or which latent features contributed most to y .
 - Map attributions to domain-interpretation: e.g., “harmonics at 5th/7th order had high positive contribution → indicates harmonic distortion type”.
4. Generate an explanation report:
 - Which features strongly influenced classification.
 - Whether input falls into known patterns (e.g., voltage sag vs swell).
 - Possibly counterfactual: what change in signal would lead to different classification.
5. Evaluate explanation quality: check fidelity (does the explanation reflect model’s decision), robustness (different similar disturbances produce similar explanations), comprehensibility (engineer can understand).
6. Use explanations for operational action: e.g., alarm, mitigation, root-cause analysis.

5. Example scenario

Suppose the classifier f processes a voltage waveform and labels it as “Voltage Sag”. The XAI module computes SHAP values over extracted features and finds:

- High positive attribution to “RMS drop in phase A for 0.15 s duration”.
- Moderate attribution to “harmonic content increase 11th order”.
- Low attribution to “swell features”.

Interpretation: The model found the short deep drop in RMS on phase A, consistent with sag. The attribution helps the operator understand *why* the model flagged sag not swell or interruption. The operator can then inspect phase A equipment, loads, connections.

6. Key benefits and considerations for PQD classification with XAI

Benefits:

- Enhances trust and acceptance among power-system engineers.
- Aids faster root-cause diagnosis (explanation points to relevant features).

- Supports compliance, auditing, and system validation.

Considerations / challenges:

- Signal data is high-dimensional and non-stationary; explanations may be complex.
- Feature extraction workflows may hide domain semantics; mapping attributions back to physical phenomena requires domain knowledge.
- Class imbalance: some disturbance types are rare; explanation might be less stable for rare classes.
- Real-time constraints: generating explanations must meet operational timing.
- Explanation evaluation: assessing whether the explanation is *valid* involves domain metrics (e.g., operator agreement) in addition to fidelity.

7. Recent research and developments

- The latent-space XAI method for PQD classifiers: builds encoder/decoder to map feature importance back to original signal space (Machlev et al.). [Technion CRIS](#)
- A study measuring explainability and trustworthiness of PQD classifier decisions using XAI. [TalTech+1](#)
- Hybrid models (CNN-LSTM) with attention mechanisms for PQD classification have been proposed; integrating XAI into such architectures is a current research direction. [MDPI+1](#)

If you like, I can **prepare a diagram** of the workflow (signal → classifier → XAI → operator) and **draft an assignment-ready answer** for you (structured, bullet-points, references).



4. Outline methods for measuring human intelligence in XAI contexts.

Here are key methods used to **measure human intelligence or human-related capacities** in the context of evaluating XAI systems (i.e., how humans understand, use, and collaborate with AI explanations). These metrics help assess whether the explanations lead to better human-AI performance, trust, and acceptance.

Methods for measurement

1. Task performance / decision outcome metrics

- Measure how well human users perform a task (e.g., classification, prediction, decision making) *with* the aid of the AI + explanation versus without. For example: accuracy, speed, error-rate, number of correct decisions. [MDPI+1](#)
- Provides objective evidence that the explanation improves human performance (an aspect of “intelligence” in interaction with AI).
- Example: “Human + AI decision accuracy improved from 72 % to 85 % when explanation was provided.”
- Important: Must be measured with the right control conditions (human alone, AI alone, human+AI+explanation). [arXiv](#)

2. Mental model / comprehension assessment

- Assess how well the user understands the AI system’s workings, logic, or explanation (i.e., the user’s internal model).
- Methods: surveys, quizzes, mapping tasks where users explain how the AI arrived at an output, or predict AI behaviour on new inputs.
- Example metric: “Percentage of user predictions about AI behaviour that are correct.”
- This addresses human intelligence in terms of insight, reasoning, and prediction. [Frontiers+1](#)

3. Trust, reliance, and acceptance metrics

- Measure user attitudes: how much they trust the AI system, whether they rely on its recommendations, whether they accept or adopt it.
- Methods: Likert-scale surveys (e.g., “I feel confident in the AI’s decisions”), behavioural measures (how often they follow AI’s advice).
- While attitude-based, these reflect human capacity to judge, engage with and integrate AI outputs. [PMC](#)

4. Cognitive load & usability metrics

- Measure how much mental effort the user requires to use or understand the explanation (a factor of human intelligence/efficiency).
- Methods: NASA-TLX (Task Load Index), time-to-decision, error counts, eye-tracking, think-aloud protocols.

- Lower cognitive load paired with good decision outcomes suggests the explanation supports human intelligence. [Frontiers](#)

5. Explanation satisfaction / subjective understanding

- Measure how satisfied users are with the explanation: clarity, informativeness, actionability.
- Methods: questionnaires post-interaction asking users to rate: “The explanation helped me understand why the AI made that decision.”
- These reflect human reflective capacity and willingness to engage. [MDPI](#)

6. Consistency / Stability of human responses

- Measure whether human decisions remain consistent when given the same explanation or similar inputs, and whether explanation influences stable decision patterns.
- Methods: repeated tasks with variation, analysing variance in responses.
- Helps assess human reasoning reliability in context of explanation.

Additional remarks

- It's critical that these human-centric measurements are **empirical** (user studies) and not only technical metrics. Many XAI studies emphasise this gap. [Frontiers+1](#)
- Selecting the right metric depends on user group (novice vs expert) and task context. For example: novices may prioritize *understanding*, experts may focus on *system performance*. [arXiv](#)
- One must distinguish between *computer-centred* metrics (fidelity, sparsity) and *human-centred* metrics (performance, trust). Both matter for human intelligence in XAI.
- Avoid relying solely on **proxy tasks** (e.g., “predict what the AI would do”) because they may not map to actual decision tasks or human intelligence in real contexts. [arXiv](#)

If you like, I can **compile a table** of these methods with typical measurement instruments (surveys, tasks, metrics) plus **example scales/questions** drawn from the literature for your assignment use.



5. Evaluate AI systems for trust, including transparency and fairness.

Here's a structured, exam-ready evaluation of how to assess AI systems for **trust**, focusing on transparency and fairness among other pillars.

1. Framework for Trustworthy AI Evaluation

Before diving into transparency and fairness specifically, it helps to use a broader framework so you know where those fit. A recent survey identifies six core requirements for trustworthy AI:

1. Human agency & oversight
2. Fairness & non-discrimination
3. Transparency & explainability
4. Robustness & accuracy
5. Privacy & security
6. Accountability [arXiv+1](#)

Thus transparency and fairness are central but interlinked with other dimensions.

2. Evaluating Transparency

Transparency means that the AI system, its model logic, data, assumptions, and decision path are sufficiently open or understandable so that stakeholders can monitor, scrutinise and trust it.

Key aspects & evaluation methods

- **Explainability:** Does the system provide explanations of its decisions that are understandable to users?
 - Metrics: explanation completeness (does it cover all relevant parts of decision), explanation fidelity (how well explanation matches model behaviour).
- **Model auditing and documentation:** Are model assumptions, training data, performance, drift behaviour documented and accessible?
 - Evaluation: checklist compliance (e.g., Model cards, Datasheets).
- **Visibility of processes:** Are users or auditors able to trace data flows, model inputs/outputs, decision logging?
 - Evaluation: logging completeness, traceability metrics.
- **Stakeholder communication:** Are explanations tailored to different audiences (technical, non-technical)?
 - Evaluation: user surveys of clarity, comprehension.

Example metric

From trustworthiness literature: measure how many decisions a user understands post explanation, or the time taken to understand the decision logic with and without explanation. [ScienceDirect+1](#)

Trade-offs and caveats

- Full transparency may conflict with proprietary models or privacy.
- Transparent logic doesn't guarantee fairness or correct outcomes.
- Users may misunderstand or mis-interpret explanations if not designed well.

3. Evaluating Fairness

Fairness means that the AI system does not discriminate unfairly across individuals or groups and treats them equitably given the context.

Key aspects & evaluation methods

- **Group fairness:** Compare model outcomes among demographic groups (e.g., gender, race, age).
 - Metrics: demographic parity difference, equal opportunity difference, disparate impact ratio. [Shelf](#)
- **Individual fairness:** Similar individuals (in terms of relevant features) should receive similar outcomes.
 - Metrics: average difference in predictions for similar pairs, Lipschitz continuity approximations.
- **Fairness through process:** Did training data, feature selection, algorithmic choices embed bias?
 - Evaluation: audit of data distributions, bias in feature importance, representation of minority groups.
- **Fairness over lifecycle:** Monitor fairness post-deployment for drift, feedback loops, amplification.
 - Evaluation: periodic fairness audits, monitoring fairness metric trends.

Example metric

A fairness metric is the **equal opportunity difference**: $P(\hat{Y} = 1 \mid Y = 1, A = 0) - P(\hat{Y} = 1 \mid Y = 1, A = 1)$, where A is protected attribute. A value close to 0 implies similar true positive rate across groups. [Iterate](#)

Trade-offs and caveats

- Improving fairness (reducing bias) may reduce model accuracy for some groups—a trade-off between fairness and performance. [MDPI](#)
- Fairness definitions conflict: demographic parity vs equal opportunity vs calibration. You may fix one and violate another.
- Fairness metrics must be contextualised—what is “fair” depends on domain, norms, regulations.

- Data bias and feature correlation complicate fairness measurement.

4. Combined Evaluation of Transparency + Fairness for Trust

When you evaluate AI systems for trust, you need to consider both transparency and fairness together.

Steps for combined evaluation

1. Define stakeholder groups and trust expectations (users, regulators, affected individuals).
2. Collect transparency metrics (explanation quality, documentation, traceability) and fairness metrics (aggregate and individual).
3. Use auditing tools and logs to assess decision consistency across groups and over time.
4. Use user-centred evaluation: how do users perceive fairness and clarity? Surveys or tasks measuring trust, comprehension, perceived fairness.
5. Identify trade-offs: e.g., if you open model logic widely you expose proprietary or privacy risks—how to handle? Or if you adjust model to be more fair, what is performance cost and how does transparency explain that?
6. Monitor over lifecycle: Trust is not one-time. Deployment, feedback loops, drift, and change of input distributions may degrade fairness and transparency.

Example frameworks

- Use frameworks such as National Institute of Standards and Technology (NIST) AI Risk Management Framework, or the “AI Trust Framework & Maturity Model (AI-TMM)”, to structure evaluation across dimensions. [MDPI+1](#)
- Conduct regular “trustworthiness health checks” that include transparency audit + fairness metrics + user trust surveys.

5. Practical Checklist for Your Assignment

- **Transparency:**
 - Does the system provide human-readable explanations for decisions?
 - Are model assumptions, training data, versioning, decision logs documented?
 - Is there traceability from input to output?
 - Are explanations validated for fidelity and comprehension?
- **Fairness:**
 - Are protected groups identified?
 - Are model outcomes compared across groups (true positives, false negatives)?
 - Are bias mitigation strategies in place?
 - Is there monitoring over time for fairness drift?

- **Trust:**
 - Aggregate transparency plus fairness plus performance plus user perceptions.
 - Do users trust the system? (Surveys, behavioural metrics)
 - Does the system allow human override and accountability?
 - Is there governance, stakeholder engagement, auditability?
-

6. Summary

Evaluating AI systems for trust requires doing more than checking accuracy. Two key pillars are **transparency** and **fairness**. Transparency ensures users and auditors understand how decisions are made; fairness ensures decisions don't systematically discriminate. Together they form major components of the broader trustworthy-AI framework. Without them trust will degrade even if performance is high.

If you like, I can **draft a full answer** formatted for a 6-mark exam (with introduction, body, conclusion) and include **real-world case-study bullet points** (e.g., biased hiring algorithm, transparent medicine AI) to enrich it.